

Restricted Kinetic Balance (RKB): Equations and Implementation

Analytic construction, spherical large-component reduction, and the LOWGEN safety net

1. Kinetic balance

The one-electron Dirac equation couples a large-component spinor ψ_L and a small-component spinor ψ_S through

$$\psi_S = \frac{1}{2c} \boldsymbol{\sigma} \cdot \mathbf{p} \psi_L \quad (\text{restricted kinetic balance, RKB}), \quad (1)$$

with $\mathbf{p} = -i\nabla$ and $\boldsymbol{\sigma} = (\sigma_x, \sigma_y, \sigma_z)$ the Pauli matrices. A finite-basis 4-component calculation must choose a small-component AO basis that reproduces this relation *exactly* for every large-component basis function; otherwise the calculation suffers from “variational collapse” into spurious negative-energy states. The overall factor $1/2c$ is a normalization convention absorbed into `Utils/DiracKinetic.h`’s own matrix elements and plays no role below – the only thing that must be built *exactly* is the AO-basis image of $\boldsymbol{\sigma} \cdot \mathbf{p} \psi_L$ for every large function ψ_L .

This project builds that image *analytically*, by literal differentiation of each large-component Cartesian Gaussian, rather than numerically (via an unrestricted kinetic-balance basis projected down by a generalized inverse) as earlier versions of this code did.

2. Differentiating a Cartesian Gaussian

A contracted large-component Cartesian AO is

$$\chi(\mathbf{r}) = \sum_k c_k x^{l_x} y^{l_y} z^{l_z} e^{-a_k r^2}, \quad (2)$$

where the sum runs over primitives k (exponent a_k , libcint-convention coefficient c_k ; see §3) and $l_x + l_y + l_z = l$. Differentiating one Cartesian axis gives, exactly, a *lowering* and a *raising* piece:

$$\frac{\partial}{\partial x} \left[x^{l_x} y^{l_y} z^{l_z} e^{-a_k r^2} \right] = \underbrace{l_x x^{l_x-1} y^{l_y} z^{l_z} e^{-a_k r^2}}_{\text{lowering (absent if } l_x=0)} - \underbrace{2a_k x^{l_x+1} y^{l_y} z^{l_z} e^{-a_k r^2}}_{\text{raising (always present)}}, \quad (3)$$

and likewise for $\partial/\partial y$, $\partial/\partial z$. Each of the (up to) six resulting elementary terms per large function is itself an ordinary Cartesian Gaussian AO and is appended to the RKB small-component basis (`RKB/RkbDerivativeTerms.cpp`, `buildRkbSmallBasis`). Because this is a literal calculus identity, *no* overlap matrix inversion or least-squares projection is needed to recover the expansion – the small-component partner of every large function is built directly, in closed form, and no two large functions ever share an elementary small-component term (unlike the pooled unrestricted-kinetic-balance scheme this replaced).

2.1 The libcint normalization correction

The naive weights above (l_x for lowering, $-2a_k$ for raising) are only correct if c_k is a literal coefficient multiplying the bare monomial $\times$ Gaussian. In practice c_k is stored in *libcint's own convention*, which applies a hidden, $(l, \text{component})$ -dependent internal scale (via `CINTgto_norm`) that plain differentiation does not know about: changing a term's Cartesian type from $(l, \mathbf{c}_{\text{old}})$ to $(l \pm 1, \mathbf{c}_{\text{new}})$ at the same exponent a requires an additional correction factor

$$\text{weight}_{\text{libcint}} = \text{weight}_{\text{literal}} \times \frac{\kappa(l_{\text{new}}, \mathbf{c}_{\text{new}}, a)}{\kappa(l_{\text{old}}, \mathbf{c}_{\text{old}}, a)}, \quad \kappa(l, \mathbf{c}, a) = \sqrt{\frac{S_{\text{true}}(\mathbf{c}, a)}{S_{\text{raw}}(l, \mathbf{c}, a)}}, \quad (4)$$

where $S_{\text{raw}}(l, \mathbf{c}, a)$ is libcint's own raw self-overlap of a unit-coefficient primitive of Cartesian type $\mathbf{c} = (l_x, l_y, l_z)$ (`Integrals.h's shellSelfOverlap`, the same machinery `normalizeCartesianBasis` itself uses), and S_{true} is the ordinary, convention-free mathematical self-overlap of the bare monomial $\times$ Gaussian,

$$S_{\text{true}}(l_x, l_y, l_z, a) = M(l_x, a) M(l_y, a) M(l_z, a), \quad M(n, a) = \frac{(2n-1)!!}{(4a)^n} \sqrt{\frac{\pi}{2a}} \quad (5)$$

(the elementary 1-D Gaussian moment $\int x^{2n} e^{-2ax^2} dx$). This correction factor is computed *directly* from already-validated integral machinery, not from an assumed formula for libcint's internal convention – confirmed empirically to resolve a multi-Hartree discrepancy (a ratio of exactly $\sqrt{3}$ for an $s \rightarrow p$ transition) down to machine precision against the independent `nablaIntegral` ground truth.

3. Assembling the RKB expansion coefficients

Write $D_k(p)$ for the pair of elementary terms (lowering, raising) that differentiation along axis $k \in \{x, y, z\}$ produces for large function p , each already carrying its own closed-form, libcint-corrected coefficient from §2. The restricted-kinetic-balance relation $\psi_S \propto \boldsymbol{\sigma} \cdot \mathbf{p} \psi_L$ expands, in the 2-spinor (large-alpha, large-beta) $\times$ (small-alpha, small-beta) block structure, as

$$\boldsymbol{\sigma} \cdot \mathbf{p} = \begin{pmatrix} p_z & p_x - ip_y \\ p_x + ip_y & -p_z \end{pmatrix}, \quad p_k = -i \partial_k, \quad (6)$$

giving the four RKB coefficient blocks (`RKB/RkbTransformation.cpp`, cross-validated two independent ways: against this project's own earlier numerically-projected construction via an integration-by-parts identity, and directly against $\boldsymbol{\sigma} \cdot \mathbf{p}$ applied to an explicit two-component spinor):

$$C_{(\text{L}\alpha, \text{S}\alpha)} = -i D_z(p), \quad C_{(\text{L}\alpha, \text{S}\beta)} = -i D_x(p) + D_y(p), \quad (7)$$

$$C_{(\text{L}\beta, \text{S}\alpha)} = -i D_x(p) - D_y(p), \quad C_{(\text{L}\beta, \text{S}\beta)} = +i D_z(p). \quad (8)$$

Collected over every large function p , this is the $(2n_L) \times (2n_S)$ matrix C returned by `rkbCoefficients`, satisfying $\boldsymbol{\sigma} \cdot \mathbf{p} |\text{Large}_p\rangle = \sum_q C_{pq} |\text{Small}_q\rangle$ exactly, with n_S the raw analytic small-basis size (at most $6n_L$, before the reduction of §4–§5).

3.1 Concrete assembly

The $D_k(p)$ notation above is shorthand, not a separate object to compute: it stands for whichever elementary small-basis function(s) §2 built for large function p along axis k , each *already carrying*

its own closed-form, libcint-corrected coefficient (l_k or $-2a_i$, times the §2.1 correction) inside that small basis function itself – that weight is never stored in C . C therefore stores only the bare Pauli-matrix phase, placed at whichever column index(es) §2’s bookkeeping (`RkbDerivativeTerms`: six slots `lower_x`, `raise_x`, `lower_y`, `raise_y`, `lower_z`, `raise_z` per large function, `lower_k` absent/omitted whenever $l_k = 0$ already) assigned to those terms. Writing $q_k^-(p)/q_k^+(p)$ for p ’s lowering/raising column index along axis k (omitted if absent), row p of each spin block of C has at most six nonzero entries, read directly off the four formulas above:

$$\text{Large-}\alpha \text{ row, Small-}\alpha \text{ block: } C_{p,q_z^\mp(p)} = -i, \quad (9)$$

$$\text{Large-}\alpha \text{ row, Small-}\beta \text{ block: } C_{p,q_x^\mp(p)} = -i, \quad C_{p,q_y^\mp(p)} = +1, \quad (10)$$

$$\text{Large-}\beta \text{ row, Small-}\alpha \text{ block: } C_{p,q_x^\mp(p)} = -i, \quad C_{p,q_y^\mp(p)} = -1, \quad (11)$$

$$\text{Large-}\beta \text{ row, Small-}\beta \text{ block: } C_{p,q_z^\mp(p)} = +i, \quad (12)$$

(each “ $q_k^\mp(p)$ ” standing for *two* entries, at $q_k^-(p)$ and $q_k^+(p)$ separately, both carrying the same phase) and every other entry of row p is exactly zero – C is sparse by construction, never assembled as, or requiring the inversion of, a dense overlap matrix. This is the entirety of `rkbCoefficients`’s loop body (`RKB/RkbTransformation.cpp`).

Worked example (why C is just $\pm i, \pm 1$). Take the (Large- α , Small- α) block, $p_z = -i\partial_z$ acting on $\text{Large}_p = x^{l_x}y^{l_y}z^{l_z}e^{-ar^2}$:

$$-i\partial_z \text{Large}_p = -i \left[\underbrace{l_z x^{l_x}y^{l_y}z^{l_z-1}e^{-ar^2}}_{\text{lowering piece}} - \underbrace{2a x^{l_x}y^{l_y}z^{l_z+1}e^{-ar^2}}_{\text{raising piece}} \right]. \quad (13)$$

The two bracketed pieces are *not* bare monomials still waiting to be scaled – by §2’s own construction, $\text{Small}_{q_z^-(p)}$ *is*, as a stored basis function, exactly $l_z x^{l_x}y^{l_y}z^{l_z-1}e^{-ar^2}$ (times the §2.1 libcint correction), and $\text{Small}_{q_z^+(p)}$ *is* exactly $-2a x^{l_x}y^{l_y}z^{l_z+1}e^{-ar^2}$ (times the same correction) – the weight was baked into the object when §2 built it, never left as a separate factor to apply later. So the identity above reads, with no further scaling needed,

$$-i\partial_z \text{Large}_p = (-i) \cdot \text{Small}_{q_z^-(p)} + (-i) \cdot \text{Small}_{q_z^+(p)}, \quad (14)$$

i.e. $C_{p,q_z^-(p)} = C_{p,q_z^+(p)} = -i$ exactly, matching the table above. §2 (the basis functions) and C (this matrix) divide the calculus between them by construction: §2 carries every quantitative weight (l_k , $-2a_k$, the libcint correction), C carries only which spin block gets which axis’s derivative, with the bare Pauli phase – which is also exactly why C needs no numerical work (projection, inversion, least squares) to write down.

The RKB small-component overlap and one-electron matrices follow from C by the congruence

$$S_{\text{small}} = \overline{C} S_{\text{uKB}} C^T \quad (\text{RKB/RkbOverlap.h's rkbSmallOverlapMatrix}), \quad (15)$$

where S_{uKB} (shape $2n_S \times 2n_S$) is the ordinary Cartesian AO overlap of the raw analytic small basis, and $\overline{C}$ is the plain (elementwise) complex conjugate of C – *not* $C^\dagger$: this is *not* the usual $C^\dagger S C$ congruence pattern (conjugate-transpose on one side, plain C on the other), but conjugate-only on the left and transpose-only on the right, so that the shapes work out ($2n_L \times 2n_S$, then $\times 2n_S \times 2n_S$, then $\times 2n_S \times 2n_L$, giving $2n_L \times 2n_L$). §6 explains where this convention actually comes from.

4. Large-component spherical reduction

For angular momentum $l \geq 2$ a Cartesian shell contains redundant combinations: 6 Cartesian d 's for 5 genuine spherical harmonics, 10 f 's for 7, and so on ($\frac{(l+1)(l+2)}{2}$ Cartesian components vs. $2l+1$ spherical ones). DIRAC itself, and this project's own `Utils/SphericalTransform.h`, work in the spherical (real-solid-harmonic) large-component basis for exactly this reason.

The genuine angular-momentum- l subspace is the *harmonic* one: polynomials annihilated by the Laplacian. Writing $\mathcal{L}_l$ for the exact, small-integer linear map from degree- l monomials to degree- $(l-2)$ monomials induced by $\partial_x^2 + \partial_y^2 + \partial_z^2$, the harmonic subspace is $\ker \mathcal{L}_l$, of dimension exactly $2l+1$ (a standard fact: $\dim \ker \mathcal{L}_l = \frac{(l+1)(l+2)}{2} - \frac{(l-1)l}{2} = 2l+1$). Since $\ker \mathcal{L}_l = \ker(\mathcal{L}_l^\top \mathcal{L}_l)$, this null space is found by diagonalizing the symmetric, positive-semidefinite matrix

$$\mathcal{L}_l^\top \mathcal{L}_l = U \operatorname{diag}(\mu_1 \leq \mu_2 \leq \dots) U^\top, \quad N := U_{[:,1:2l+1]}, \quad (16)$$

i.e. N is the $\frac{(l+1)(l+2)}{2} \times (2l+1)$ matrix of eigenvectors belonging to $\mathcal{L}_l^\top \mathcal{L}_l$'s $2l+1$ zero eigenvalues (exact, machine-precision zeros, since $\mathcal{L}_l$ has only small-integer entries, with an $O(1)$ gap to every remaining eigenvalue) – `SphericalTransform.cpp`'s `harmonicNullSpaceBasis`. Since eigenvectors of a symmetric matrix are orthonormal, $N^\top N = I_{2l+1}$ already; N is an orthonormal basis of the harmonic subspace in the Euclidean sense only, not yet under the physical Gaussian overlap metric. Löwdin-orthonormalizing N under the shell's own physical overlap metric $\bar{S}_l$ – a small, *universal* reference matrix depending only on l (e.g. 6×6 for a d -shell), computed once from an arbitrary reference exponent/coefficient, since a shell's self-overlap is always a scalar multiple of it regardless of the actual molecule or basis set (*not* to be confused with S_{uKB} of §3/§6, the real, molecule-specific small-component AO overlap) – gives the final transform block

$$T_l = N (N^\top \bar{S}_l N)^{-1/2} \quad \left(\text{an } \frac{(l+1)(l+2)}{2} \times (2l+1) \text{ matrix, } T_l^\top \bar{S}_l T_l = I \right), \quad (17)$$

computed once per angular momentum and cached. The molecule's actual Cartesian large-component basis is an ordered list of shells $s = 1, \dots, n_{\text{shells}}$, each with its own angular momentum l_s (possibly repeated: a basis typically has several s -shells, several p -shells, ...); the full Cartesian-to-spherical transform is simply these shells' own T_{l_s} blocks placed block-diagonally, in shell order, reusing the same T_l matrix wherever two shells share an l :

$$T_{\text{sph}} = \operatorname{blockdiag}(T_{l_1}, T_{l_2}, \dots, T_{l_{n_{\text{shells}}}}) \quad (\text{buildSphericalTransform}), \quad (18)$$

an $n_{\text{large, cart}} \times n_{\text{large, sph}}$ matrix overall ($n_{\text{large, sph}} = \sum_s (2l_s + 1) \leq n_{\text{large, cart}} = \sum_s \frac{(l_s+1)(l_s+2)}{2}$, equal only if every shell has $l_s \leq 1$). Validated to machine precision ($\max |T_l^\top \bar{S}_l T_l - I| \sim 10^{-15}$) for every angular momentum libcint supports, $l = 0, \dots, 6$.

5. The LOWGEN safety net

On top of the exact spherical reduction, DIRAC's own LOWGEN routine (`dirone.F`) applies a further, numerically-triggered canonical (Löwdin) orthogonalization: diagonalize the overlap $S = U \operatorname{diag}(w) U^\top$ and keep only

$$X = U_{\text{kept}} \operatorname{diag}(w_{\text{kept}}^{-1/2}), \quad \text{kept} = \{i : w_i > \text{STOL}\}, \quad (19)$$

genuinely *dropping* columns (not inverting them) whenever an eigenvalue falls at or below the threshold – $\text{STOL}(1) = 10^{-6}$ for the large component, $\text{STOL}(2) = 10^{-8}$ for the

small, matching DIRAC's own defaults. This project's `LinearAlgebra/LinearAlgebra.h`'s `canonicalOrthogonalize/ canonicalOrthogonalizeHermitian` implement exactly this generic $S \mapsto X$ map. Define X_{LOWGEN} as this map applied, specifically, to the *already-spherical* large overlap at threshold $\text{STOL}(1)$,

$$S_{\text{large,sph}} = T_{\text{sph}}^\top S_{\text{large,cart}} T_{\text{sph}}, \quad X_{\text{LOWGEN}} := X(S_{\text{large,sph}}, \text{STOL}(1)), \quad (20)$$

and combine it with T_{sph} into one transform,

$$T_{\text{final}} = T_{\text{sph}} X_{\text{LOWGEN}}, \quad (21)$$

so that the large-component dimension only shrinks further than the exact spherical count when a *genuine* residual near-linear-dependence is found (confirmed to be a no-op, bit-identical to T_{sph} alone, for every basis tested so far).

6. Keeping the small-component dimension matched

Recall from §3: C is the $(2n_L) \times (2n_S)$ matrix with $\sigma \cdot \mathbf{p} |\text{Large}_p\rangle = \sum_q C_{pq} |\text{Small}_q\rangle$, built one small partner per *large* function, n_L being whatever large-component count is currently in play (Cartesian, or already spherical/LOWGEN-reduced – the construction in §2–§3 does not care). This section explains why reducing n_L via T_{final} automatically reduces the *projected* RKB small-component count to match it exactly, with no separate bookkeeping.

6.1 The raw (uKB) one-electron Hamiltonian

Before any RKB projection, the one-electron Hamiltonian lives on the direct sum of the Large-component AO space (dimension $2n_L$, both spins) and the *raw analytic* small basis of §2 (dimension $2n_S$, both spins), with block structure (`UKB/UkbHamiltonian.h` = `Utils/DiracKinetic.h` + `Vext/Vext.h`):

$$H_{\text{uKB}} = \begin{pmatrix} V_{\text{LL}} & T_{\text{LS}} \\ T_{\text{LS}}^\dagger & V_{\text{SS}} - 2c^2 S_{\text{uKB}} \end{pmatrix}, \quad \text{dimension } (2n_L + 2n_S) \times (2n_L + 2n_S), \quad (22)$$

$V_{\text{LL}}/V_{\text{SS}}$ the nuclear attraction over the Large/raw-Small AOs respectively, T_{LS} the Large–Small kinetic coupling built from D_x, D_y, D_z (§2) with exactly the same $\sigma \cdot \mathbf{p}$ weights as §3, and $-2c^2 S_{\text{uKB}}$ the small-component rest mass term (S_{uKB} as in §3).

6.2 The RKB embedding, and its tautological dimension-matching property

Given *any* coefficient matrix C of shape $(2m) \times (2n_S)$ (not yet assumed equal to the C of §3), define its RKB embedding (`RKB/RkbHamiltonian.h`'s `rkbEmbeddingMatrix`)

$$W(C) = \begin{pmatrix} I_{2m} & 0 \\ 0 & C^T \end{pmatrix}, \quad \text{shape } (2m + 2n_S) \times 4m, \quad (23)$$

and the RKB-projected Hamiltonian $H_{\text{RKB}} = W(C)^\dagger H_{\text{uKB}} W(C)$. Since $(C^T)^\dagger = \overline{C}$ (transposing C^T undoes the transpose, then the outer dagger conjugates), $W(C)^\dagger$'s bottom-right block is $\overline{C}$, not $C^\dagger$ – exactly the $\overline{C}(\cdot)C^T$ pattern of §3's S_{small} formula, which is in fact nothing but H_{RKB} 's own Small-Small block with S_{uKB} in place of H_{uKB} 's Small-Small block. The crucial, purely definitional fact is: **$W(C)$'s own output dimension is $4m - 2m$ for the (unchanged) Large legs, and *exactly* $2m$ again for the RKB-compressed Small legs, *regardless of* n_S .** Whatever large-orbital count m the rows of C happen to represent, that is what the compressed Small-component dimension comes out as, by construction – not because of any additional reduction step on the Small side.

6.3 Applying this with the reduced large dimension

§3's C has $m = n_{\text{L, cart}}$ (one partner per *Cartesian* large function). Feeding it directly into $W(\cdot)$ would therefore give a Small-component count of $n_{\text{L, cart}}$, not the smaller $n_{\text{L, final}}$ that §4–§5 reduced the Large side to. The fix is to row-project C through the *same* T_{final} *before* forming W , so that its row count already reflects the reduced Large dimension. Spin-duplicating T_{final} block-diagonally across the α, β Large spin-orbitals,

$$T_{\text{spin}} = \begin{pmatrix} T_{\text{final}} & 0 \\ 0 & T_{\text{final}} \end{pmatrix} \quad (\text{shape } 2n_{\text{L, cart}} \times 2n_{\text{L, final}}), \quad (24)$$

define the embedding that reduces *only* the Large legs of H_{uKB} , leaving the raw analytic Small legs completely untouched,

$$V = \begin{pmatrix} T_{\text{spin}} & 0 \\ 0 & I_{2n_{\text{S}}} \end{pmatrix} \quad (\text{shape } (2n_{\text{L, cart}} + 2n_{\text{S}}) \times (2n_{\text{L, final}} + 2n_{\text{S}})), \quad (25)$$

and apply it to both H_{uKB} and C (`main.cpp`'s `X2C/C4_SPINOR` construction block, *before* any RKB projection happens):

$$H'_{\text{uKB}} = V^\dagger H_{\text{uKB}} V \quad (\text{shape } (2n_{\text{L, final}} + 2n_{\text{S}})^2), \quad C' = T_{\text{spin}}^\dagger C \quad (\text{shape } 2n_{\text{L, final}} \times 2n_{\text{S}}). \quad (26)$$

Raw C connects Cartesian large $\leftrightarrow$ Cartesian small (both sides of §2–§3 are built from Cartesian differentiation); C' connects spherical large $\leftrightarrow$ Cartesian small – the row-projection changes only which large-component combinations index the rows, never touching the (deliberately never spherically transformed) small/RKB side. In the actual code, `rkb_coefficients` holds C only momentarily and is overwritten in place by C' before any downstream use, so everything from here on (S_{small} , H_{RKB} , the two-electron integrals) is built from C' , never the raw C .

Now form the RKB embedding of §6.2 with $m = n_{\text{L, final}}$, i.e. with C' in place of the original C :

$$H_{\text{RKB}} = W(C')^\dagger H'_{\text{uKB}} W(C') \quad (\text{shape } 4n_{\text{L, final}}). \quad (27)$$

By §6.2's tautological property, this Hamiltonian's Small-component block is *exactly* $2n_{\text{L, final}}$ -dimensional – matching the Large block automatically, with no $n_{\text{large}} \neq n_{\text{small}}$ plumbing anywhere downstream. The identical mechanism (sandwich the Large legs with T_{final} , feed the row-projected C' into the same RKB projection) applies to the two-electron Coulomb integrals: each AO Cholesky vector's (or dense tensor's) Large legs are transformed by T_{final} before the RKB-Small projection (`C4.DHF/RkbCholesky.h`, `RkbTwoElectron.cpp`).

7. Summary of the pipeline

1. Build the large-component Cartesian AO basis (`libcint`), individually normalized.
2. Differentiate each large function analytically (§2) $\rightarrow$ raw analytic RKB small basis, and the exact expansion matrix C (§3).
3. Build the large-component spherical transform T_{sph} (§4); combine with the LOWGEN safety net into T_{final} (§5).

4. Row-project $C \rightarrow C' = T_{\text{spin}}^\dagger C$ and sandwich the one-/two-electron large-component legs with T_{final} *before* the RKB projection (§6), so the Small-component dimension tracks the Large one automatically.
5. Build $S_{\text{small}} = \overline{C'} S_{\text{uKB}} C'^T$; apply the SAME LOWGEN check (§5) to S_{small} itself, independently, to catch any genuine near-degeneracy the Large-side reduction does not already explain. If none is found (every basis tested so far), $X_{\text{small}} = S_{\text{small}}^{-1/2}$ as usual. If one is found, this is DIRAC's own accepted $n_{\text{positive}} \neq n_{\text{negative}}$ case (NESH $\neq$ NPSH), not yet threaded through the rest of this project's pipeline, and the calculation stops with an explicit, actionable error rather than silently mismatching dimensions.